

Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

Причины использования

Объявление и открытие курсора

Операции с курсором

Циклы по курсору и по результатам запроса

Передача курсора клиенту

Причины использования

Курсор подразумевает итеративную обработку

- полная выборка занимает слишком много памяти
- нужна не вся выборка, но размер заранее неизвестен
- способ отдать управление выборкой клиенту
- действительно требуется построчная обработка (обычно нет)

С концепцией курсоров мы уже знакомы по теме «Архитектура. Общее устройство PostgreSQL». Там речь шла о курсорах как о возможности, предоставляемой сервером, и мы смотрели, как к этой возможности обращаться средствами SQL. Теперь поговорим о том, как использовать те же самые курсоры в языке PL/pgSQL.

Зачем вообще может возникнуть необходимость в курсорах? Декларативный SQL в первую очередь предназначен для работы с множествами строк — в этом его сила и преимущество. PL/pgSQL, как процедурный язык, должен работать со строками по одной за раз, используя явные циклы. Такой подход как раз реализуется при помощи курсоров.

Например, полная выборка может занимать слишком много места, так что приходится обрабатывать результаты по частям. Или мы получаем набор данных неизвестного заранее размера — и в процессе выборки нужно вовремя остановиться. Или есть необходимость предоставить управление выборкой клиенту.

(Однако еще раз отметим, что хотя необходимость такой построчной обработки может возникать, в большом количестве случаев ее можно заменить чистым SQL; код в итоге окажется проще и будет быстрее работать.)

Объявление и открытие

Не связанные с запросом курсорные переменные

- объявляется переменная типа `refcursor`
- конкретный запрос указывается при открытии

Связанные с запросом курсорные переменные



- при объявлении указывается запрос (возможно, с параметрами)
- при открытии указываются фактические значения параметров

Особенности

- значение курсорной переменной — имя курсора
(можно задать явно или сгенерируется автоматически)
- переменные PL/pgSQL в запросе становятся неявными параметрами
(значения подставляются при открытии курсора)
- запрос предварительно подготавливается

Как мы видели, в SQL курсор объявлялся и открывался одновременно командой DECLARE. В PL/pgSQL это два отдельных шага. Кроме того, для доступа к курсорам используются так называемые курсорные переменные, имеющие тип refcursor и, фактически, содержащие имя курсора (причем если не указывать это имя явно, PL/pgSQL сам позаботится о его уникальности).

Курсорную переменную можно объявить, не связывая ее с конкретным запросом. Тогда при открытии курсора нужно будет указать запрос.

Другой вариант — уже при объявлении переменной указать запрос, возможно, с параметрами. Тогда при открытии курсора указываются только фактические параметры.

Оба способа равноценны; какой использовать — дело вкуса.
И связанные, и несвязанные курсорные переменные инициализируются только при открытии курсора; в обоих случаях соответствующий запрос может иметь неявные параметры — переменные PL/pgSQL.

Напомним, что запрос, открытый с помощью курсора, автоматически подготавливается.

https://postgrespro.ru/docs/postgresql/16/plpgsql-cursors#PLPGSQL-CURSOR-DECLARATIONS

https://postgrespro.ru/docs/postgresql/16/plpgsql-cursors#PLPGSQL-CURSOR-OPENING

Объявление и открытие

Создадим таблицу:

```
=> CREATE TABLE t(id integer, s text);
```

```
CREATE TABLE
```

```
=> INSERT INTO t VALUES (1, 'Раз'), (2, 'Два'), (3, 'Три');
```

```
INSERT 0 3
```

Несвязанная переменная:

```
=> DO $$
DECLARE
    -- объявление переменной
    cur refcursor;
BEGIN
    -- связывание с запросом и открытие курсора
    OPEN cur FOR SELECT * FROM t;
END
$$;
```

```
DO
```

Связанная переменная: запрос указывается уже при объявлении. При этом переменная cur имеет тот же тип refcursor.

```
=> DO $$
DECLARE
    -- объявление и связывание переменной
    cur CURSOR FOR SELECT * FROM t;
BEGIN
    -- открытие курсора
    OPEN cur;
END
$$;
```

```
DO
```

В случае связанной переменной запрос может иметь параметры.

Обратите внимание на устранение неоднозначности имен в этом и следующем примерах.

```
=> DO $$
DECLARE
    -- объявление и связывание переменной
    cur CURSOR(id integer) FOR SELECT * FROM t WHERE t.id = cur.id;
BEGIN
    -- открытие курсора с указанием фактических параметров
    OPEN cur(1);
END
$$;
```

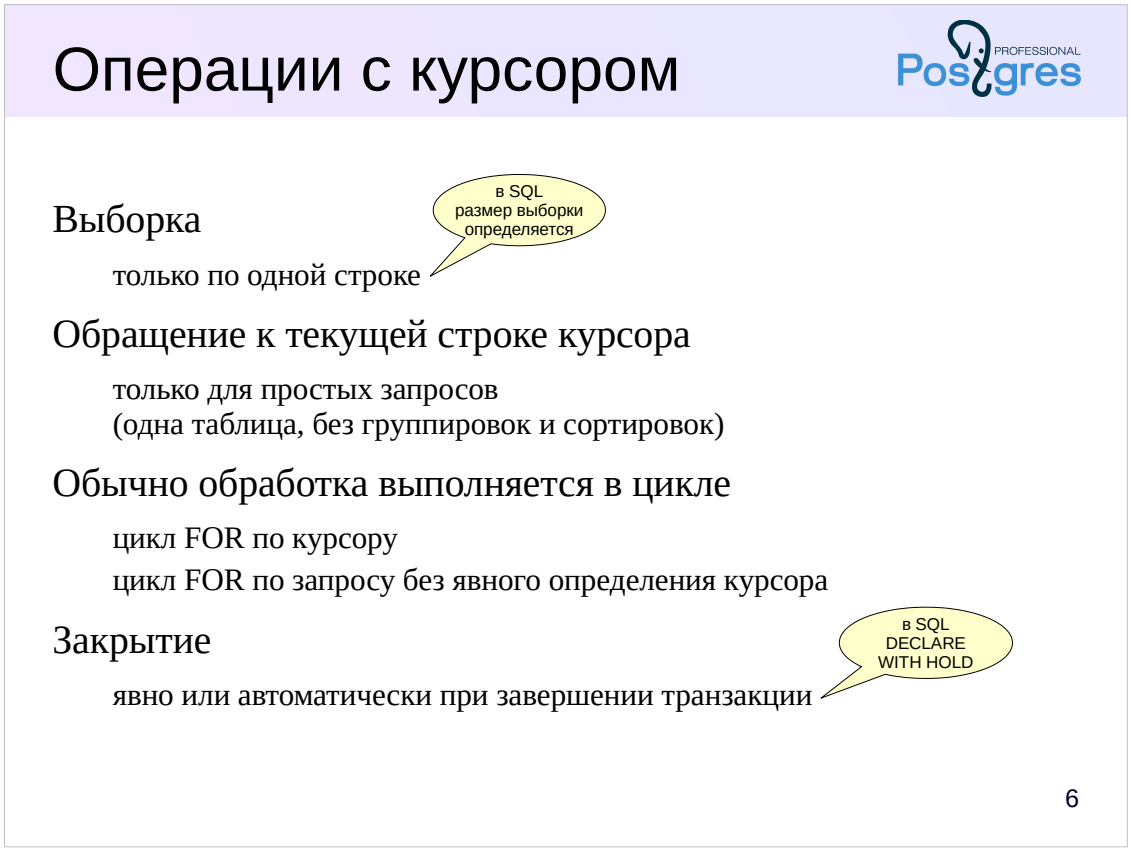
```
DO
```

Переменные PL/pgSQL также являются (неявными) параметрами курсора.

```
=> DO $$
<<local>>
DECLARE
    id integer := 3;
    -- объявление и связывание переменной
    cur CURSOR FOR SELECT * FROM t WHERE t.id = local.id;
BEGIN
    id := 1;
    -- открытие курсора (значение id берется на этот момент)
    OPEN cur;
END
$$;
```

```
DO
```

В качестве запроса можно использовать не только команду SELECT, но и любую другую, возвращающую результат (например, INSERT, UPDATE, DELETE с фразой RETURNING).



Выборка строк из курсора возможна в PL/pgSQL только по одной. Для этого служит команда FETCH INTO.

Если запрос достаточно простой (одна таблица, без группировок и сортировок), то в процессе работы с курсором можно обращаться к текущей строке, например, в командах UPDATE или DELETE.

Процедурная обработка данных подразумевает работу с циклами. Можно организовать перебор и обработку всех строк, возвращаемых курсором, с помощью тех управляющих команд, что мы уже знаем. Но, поскольку часто нужен именно такой цикл, в PL/pgSQL есть специальная команда FOR. Целочисленный вариант FOR мы уже видели в теме «Обзор и конструкции языка», а этот вариант работает с курсором. Более того, есть еще один вариант цикла FOR, в котором не требуется даже объявления курсора — в команде указывается сам запрос.

Курсор можно закрыть явно командой CLOSE, но он в любом случае будет закрыт по окончании транзакции (в SQL курсор может оставаться открытым и после завершения транзакции, если указать фразу WITH HOLD).

https://postgrespro.ru/docs/postgresql/16/plpgsql-cursors#PLPGSQL-CURSOR-USING

https://postgrespro.ru/docs/postgresql/16/plpgsql-cursors#PLPGSQL-CURSOR-FOR-LOOP

Операции с курсором

Чтение текущей строки из курсора выполняется командой FETCH. Если нужно только переместиться на следующую строку, можно воспользоваться другой командой — MOVE.

Что будет выведено на экран?

```
=> DO $$
DECLARE
    cur refcursor;
    rec record; -- можно использовать и несколько скалярных переменных
BEGIN
    OPEN cur FOR SELECT * FROM t ORDER BY id;
    MOVE cur;
    FETCH cur INTO rec;
    RAISE NOTICE '%', rec;
    CLOSE cur;
END
$$;
```

```
NOTICE:  (2,Два)
DO
```

Обычно выборка происходит в цикле, который можно организовать так:

```
=> DO $$
DECLARE
    cur refcursor;
    rec record;
BEGIN
    OPEN cur FOR SELECT * FROM t;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND; -- FOUND: выбрана ли очередная строка?
        RAISE NOTICE '%', rec;
    END LOOP;
    CLOSE cur;
END
$$;
```

```
NOTICE:  (1,Раз)
NOTICE:  (2,Два)
NOTICE:  (3,Три)
DO
```

Но чтобы не писать много команд, в PL/pgSQL имеется цикл FOR по курсору, который делает ровно то же самое:

```
=> DO $$
DECLARE
    cur CURSOR FOR SELECT * FROM t;
    -- переменная цикла не объявляется
BEGIN
    FOR rec IN cur LOOP -- cur должна быть связана с запросом
        RAISE NOTICE '%', rec;
    END LOOP;
END
$$;
```

```
NOTICE:  (1,Раз)
NOTICE:  (2,Два)
NOTICE:  (3,Три)
DO
```

Более того, можно вообще обойтись без явной работы с курсором, если цикл — это все, что требуется.

Скобки вокруг запроса не обязательны, но удобны.

```
=> DO $$
DECLARE
    rec record; -- надо объявить явно
BEGIN
    FOR rec IN (SELECT * FROM t) LOOP
        RAISE NOTICE '%', rec;
    END LOOP;
END
$$;
```

```
NOTICE:  (1,Раз)
NOTICE:  (2,Два)
NOTICE:  (3,Три)
DO
```

Как и для любого цикла, здесь можно указать метку, что может оказаться полезным во вложенных циклах:

Что будет выведено?

```
=> DO $$
DECLARE
    rec_outer record;
    rec_inner record;
BEGIN
    <<outer>>
    FOR rec_outer IN (SELECT * FROM t ORDER BY id) LOOP
        <<inner>>
        FOR rec_inner IN (SELECT * FROM t ORDER BY id) LOOP
            EXIT outer WHEN rec_inner.id = 3;
            RAISE NOTICE '%, %', rec_outer, rec_inner;
        END LOOP INNER;
    END LOOP outer;
END
$$;
```

```
NOTICE:  (1,Раз), (1,Раз)
NOTICE:  (1,Раз), (2,Два)
DO
```

После выполнения цикла переменная FOUND позволяет узнать, была ли обработана хотя бы одна строка:

```
=> DO $$
DECLARE
    rec record;
BEGIN
    FOR rec IN (SELECT * FROM t WHERE false) LOOP
        RAISE NOTICE '%', rec;
    END LOOP;
    RAISE NOTICE 'Была ли как минимум одна итерация? %', FOUND;
END
$$;
```

```
NOTICE:  Была ли как минимум одна итерация? f
DO
```

На текущую строку курсора, связанного с простым запросом (по одной таблице, без группировок и сортировок) можно сослаться с помощью предложения CURRENT OF. Типичный случай применения — обработка пакета заданий с изменением статуса каждого из них.

```
=> DO $$
DECLARE
    cur refcursor;
    rec record;
BEGIN
    OPEN cur FOR SELECT * FROM t
        FOR UPDATE; -- строки блокируются по мере обработки
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        UPDATE t SET s = s || ' (обработано)' WHERE CURRENT OF cur;
    END LOOP;
    CLOSE cur;
END
$$;
```

```
DO
```

```
=> SELECT * FROM t;
```

```
 id |        s
----+-----------------
  1 | Раз (обработано)
  2 | Два (обработано)
  3 | Три (обработано)
(3 rows)
```

Заметим, что CURRENT OF не работает с циклом FOR по запросу, поскольку этот цикл не использует курсор явным образом. Конечно, аналогичный результат можно получить, явно указав в команде UPDATE или DELETE уникальный ключ таблицы (WHERE id = rec.id). Но CURRENT OF работает быстрее и не требует наличия индекса.

Следует заметить, что в большом числе случаев вместо использования циклов можно выполнить задачу одним

оператором SQL — и это будет проще и еще быстрее. Часто циклы используют просто потому, что это более привычный, «процедурный» стиль программирования. Но для баз данных этот стиль не подходит.

Например:

```
=> BEGIN;
DO $$
DECLARE
    rec record;
BEGIN
    FOR rec IN (SELECT * FROM t) LOOP
        RAISE NOTICE '%', rec;
        DELETE FROM t WHERE id = rec.id;
    END LOOP;
END
$$;
ROLLBACK;
```

```
BEGIN
NOTICE:  (1,"Раз (обработано)")
NOTICE:  (2,"Два (обработано)")
NOTICE:  (3,"Три (обработано)")
DO
ROLLBACK
```

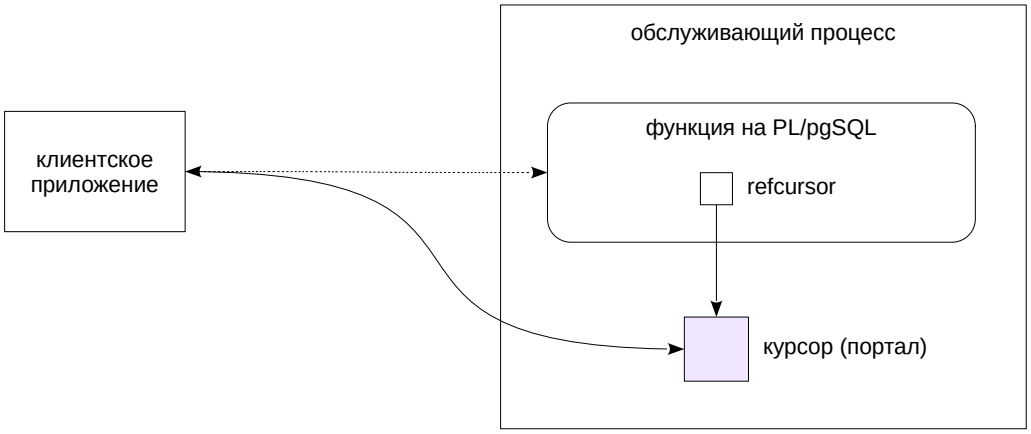
Такой цикл заменяется одной простой SQL-командой:

```
=> BEGIN;
DELETE FROM t RETURNING *;
ROLLBACK;
```

```
BEGIN
 id |        s
----+-----------------
  1 | Раз (обработано)
  2 | Два (обработано)
  3 | Три (обработано)
(3 rows)

DELETE 3
ROLLBACK
```

Передача курсора клиенту



Как уже говорилось, курсорная переменная PL/pgSQL (типа refcursor) содержит имя открытого SQL-курсора. Когда говорят о курсоре как о памяти, отведенной в обслуживающем процессе для хранения его состояния, в документации используется термин *портал*.

Таким образом, функция на PL/pgSQL может открыть курсор и вернуть его имя клиенту. Дальше клиент сможет работать с курсором так, как будто он сам его и открыл, но доступ будет иметь только к тем данным, которые предоставлены при открытии. Это дает еще одну возможность для организации взаимодействия приложения и базы данных.

Передача курсора клиенту

Откроем курсор и выведем значение курсорной переменной:

```
=> DO $$
DECLARE
    cur refcursor;
BEGIN
    OPEN cur FOR SELECT * FROM t;
    RAISE NOTICE '%', cur;
END
$$;
```

```
NOTICE:  <unnamed portal 14>
DO
```

Это имя курсора (портала), который был открыт на сервере. Оно было сгенерировано автоматически.

При желании имя можно задать явно (но оно должно быть уникальным в сеансе):

```
=> DO $$
DECLARE
    cur refcursor := 'cursor12345';
BEGIN
    OPEN cur FOR SELECT * FROM t;
    RAISE NOTICE '%', cur;
END
$$;
```

```
NOTICE:  cursor12345
DO
```

Пользуясь этим, можно написать функцию, которая откроет курсор и вернет его имя:

```
=> CREATE FUNCTION t_cur() RETURNS refcursor
AS $$
DECLARE
    cur refcursor;
BEGIN
    OPEN cur FOR SELECT * FROM t;
    RETURN cur;
END
$$ VOLATILE LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Клиент начинает транзакцию...

```
=> BEGIN;
```

```
BEGIN
```

...вызывает функцию, узнает имя курсора...

```
=> SELECT t_cur();
```

```
        t_cur
---------------------
 <unnamed portal 15>
(1 row)
```

...и получает возможность читать из него данные (кавычки нужны из-за спецсимволов в имени):

```
=> FETCH "<unnamed portal 15>";
```

```
 id |        s
----+-----------------
  1 | Раз (обработано)
(1 row)
```

```
=> COMMIT;
```

```
COMMIT
```

Для удобства можно позволить клиенту самому устанавливать имя курсора:

```
=> DROP FUNCTION t_cur();
```

```
DROP FUNCTION
```

```
=> CREATE FUNCTION t_cur(cur refcursor) RETURNS void
AS $$
BEGIN
    OPEN cur FOR SELECT * FROM t;
END
$$ VOLATILE LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Клиентский код упрощается:

```
=> BEGIN;
```

```
BEGIN
```

```
=> SELECT t_cur('cursor12345');
```

```
 t_cur
-------

(1 row)
```

```
=> FETCH cursor12345;
```

```
 id |        s
----+------------------
  1 | Раз (обработано)
(1 row)
```

```
=> COMMIT;
```

```
COMMIT
```

Функция может вернуть и несколько открытых курсоров, используя OUT-параметры. Таким образом можно за один вызов функции обеспечить клиента информацией из разных таблиц, если это необходимо.

Альтернативный подход — сразу выбрать все необходимые данные на стороне сервера и сформировать из них документ JSON или XML. Работа с этими форматами рассматривается в курсе DEV2.

Итоги

Курсор позволяет получать
и обрабатывать данные построчно

Цикл FOR упрощает работу с курсорами

Обработка в цикле естественна для процедурных языков,
но этим не стоит злоупотреблять